

<< 算法设计与分析 >>

No.

DATE / /

§1. Basis.

1. 算法.

10: 解决特定问题的有限指令序列.

→ 算法中的运算, 均可由有限论基运算实现 (每个指令必须为, 有穷性、确切性、可行性、输入、输出. 是可行的.).

e.g. 欧几里得算法. 思考: 有穷性, 无二义性.

Euclid (m, n): // 输入: 非负整数 m, n , 输出: $\gcd(m, n)$.

while $n > 0$ do

$r \leftarrow m \% n$

$m \leftarrow n$

$n \leftarrow r$

return m

! 原理: $\gcd(m, n) = \gcd(n, m \% n)$

→ 数学归纳法在 CS 中的复杂.

① 证明算法正确性:

循环不变量: 指在 Loop 每次迭代前后均为真的一性质.

证: set 不变量: $\gcd(m_i, n_i) = \gcd(m_0, n_0)$

初始化: true

保持:

数学归纳法: $\gcd(m_i, n_i)$

$= \gcd(n_i, m_i \% n_i)$

$= \gcd(m_{i+1}, n_{i+1})$

终止:

$\Rightarrow \gcd(m_0, n_0) = \gcd(m_k, n_k)$ // 共执行 k 次迭代!

$= \gcd(n_k, 0) = n_k$

初始化: before loop, true

保持: 每次迭代后, true

终止: false $\rightarrow$ 推出结果正确.

② 运行效率: 时间复杂度.

→ 屏蔽层的基本运算次数.

最坏情况下: $W(n)$

平均 ... : $A(n)$

No.

DATE / /

§ 2. 渐近分析

→ 描述收敛或发散的渐近行为

? Why need 渐近分析? 精确计算时间复杂度 $T(n)$ 十分复杂.And in fact, we only care $n \rightarrow \infty$ 时的时间增长率.

△ 5个渐符号.

符号	定义	数学定义.
O	上界	$\exists c, n_0 > 0$ s.t. $\forall n > n_0, 0 \leq f(n) \leq c \cdot g(n)$. 记作 $f(n) = O(g(n))$.
o	非紧确上界	$\forall c, \exists n_0$ s.t. $\forall n > n_0, 0 \leq f(n) < c \cdot g(n)$
Θ	紧确界	$f(n) = c \cdot g(n)$
ω	非紧确下界	$0 \leq c \cdot g(n) < f(n)$
Ω	下界	$0 \leq c \cdot g(n) \leq f(n)$

↓
渐近: $f(n) \in O(g(n))$
紧确

△ Common 复杂度.

$$1 \ll \log \log n \ll \log n \ll \sqrt{n} \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \leq n! \ll n^n$$

Stirling 公式: $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \Rightarrow \log(n!) = \Theta(n \log n)$ (pl CS中, $\log$ 的
底默认为2).

调和级数: $\sum_{k=1}^n \frac{1}{k} = \Theta(\log n)$

$$n! = o(n^n)$$

$$n! = \omega(2^n)$$

$$\Delta f(n) = o(g(n)) \iff f(n) = O(g(n))$$

e.g. $f(n) = n^2 + n$

$$f(n) = O(n^2), f(n) = O(n^3), f(n) = o(n^3), f(n) \neq o(n^2)$$

△ 排序: $2^n, n^n, n!, n \cdot 2^n, (3/2)^n, (\log n)^{\log n} = n^{\log \log n}, n^3$
(DESC) 双指数, 指数, 指数, 指数, 指数, 双对数, 多项式.

$$\log(n!) = \Theta(n \log n), n = \Theta(2^{\log n})$$

线性.

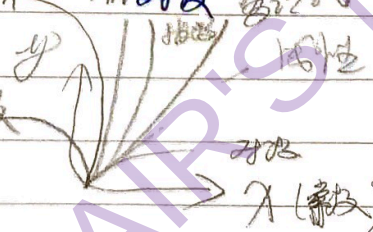
多项式, 对数, 平方根

$$\log \log n, n^{\frac{1}{\log n}} = \Theta(1)$$

双对数.

$$(2^{\log n})^{\frac{1}{\log n}} = 2$$

双指数



§3. 递归.

引例: Hanoi Tower. $T(n) = 2T(n-1) + 1$. 且 $T(1) = 1$.

$$\Rightarrow T(n) = 2^n - 1 \quad \text{推导}$$

$$\begin{aligned} \text{迭代法: } T(n) &= 2T(n-1) + 1 \\ &= 4T(n-2) + 2 + 1 \\ &= 8T(n-3) + 4 + 2 + 1 \\ &\Rightarrow T(n) = 2^k T(n-k) + \sum_{i=0}^{k-1} 2^i \\ &= 2^{n-1} T(1) + \frac{1 \cdot (1-2^n)}{1-2} \\ &= 2^n - 1. \end{aligned}$$

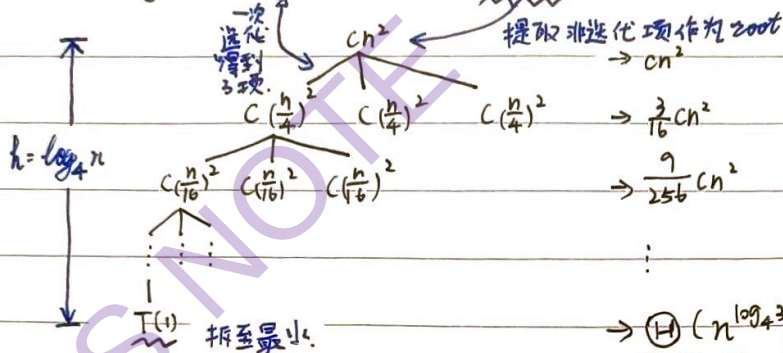
1. 递归方程的方法.

A. 迭代归纳法.

e.g. $W(n) = W(n-1) + n$ (直接划定平面, 每加一条边, 最多加 n 个边).

B. 递归树法.

e.g. $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^3)$.



$$\begin{aligned} L_1 &: 3 \times (\frac{1}{4})^2 \times cn^2 \\ L_2 &: 3^2 \times (\frac{1}{4})^2 \times cn^2 \\ L_3 &: 3^3 \times (\frac{1}{4})^2 \times cn^2 \\ &\vdots \\ L_n &: 3^n \times (\frac{1}{4})^2 \times cn^2 \\ &= (\frac{3}{16})^n \cdot cn^2 \end{aligned}$$

$$L_h = (\frac{3}{16})^{\log_4 n} \cdot cn^2$$

$$= \frac{3^{\log_4 n}}{16^{\log_4 n}} \cdot cn^2 = \frac{3^{\log_4 n}}{n^2} \cdot cn^2$$

$$= c \cdot 3^{\log_4 n} = \Theta(3^{\log_4 n})$$

$$\text{化简: } 3^{\log_4 n} = 3^{\frac{\log_2 n}{\log_2 4}} = n^{\frac{\log_2 3}{\log_2 4}} = n^{\log_4 3}$$

$$\Rightarrow \Theta(n^{\log_4 3})$$

$$\Rightarrow T(n) = \sum_{i=0}^{\log_4 n-1} (\frac{3}{16})^i \cdot cn^2 + \Theta(n^{\log_4 3})$$

$$< \sum_{i=0}^{\infty} (\frac{3}{16})^i \cdot cn^2 + \Theta(n^{\log_4 3})$$

$$= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3})$$

$$= O(n^2)$$

验证: $T(n) \leq 3 \cdot d \cdot (\frac{n}{4})^2 + cn^2$ 假设 $T(n-1)$ 满足.

$$\leq 3d \cdot (\frac{n}{4})^2 + cn^2$$

$$\leq dn^2 \Rightarrow T(n) \text{ 也满足. 确保了传递性.}$$

当 $d \geq \frac{16}{3}c$ 时, 成立.

C. 主定理.

层数 $k \approx \log_b n$.

$$T(n) = aT(n/b) + f(n). \quad (a \geq 1, b > 1) \quad \text{递归叶子节点的增长速率 } n^{\log_b a}$$

比较结果.

复杂度

理解.

$$f(n) = O(n^{\log_b a - \epsilon})$$

$$T(n) = \Theta(n^{\log_b a})$$

叶子节点 (递归部分) 太重, 占主导.

$$f(n) = \Theta(n^{\log_b a})$$

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

递归 & 非递归部分均力致.

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

$$T(n) = \Theta(f(n)), \quad \text{层数}$$

根节点占主导. (需满足正则条件: $\exists c, k$ 对 $\forall n$ 下层 k 层 $\leftarrow af(n/b) \leq cf(n)$)

No.

DATE / /

 $\Delta \epsilon$ 的作用递归部分的阶: $n^{\log_b a}$ 假设 $f(n)$ 的阶: n^k 为判断二者是否有阶的差距, 如情况三, 要求 $k > \log_b a$ 量化 $\rightarrow k = \log_b a + \epsilon$ Δ 主定理条件间的空隙

e.g. $T(n) = 2T(n/2) + n \log n$

$$\begin{cases} f(n) = n \log n \\ n^{\log_b a} = n \end{cases} \Rightarrow \text{并未大出一个多项式因子 } n^\epsilon, \text{ 只大了一个对数因子 } \log n$$

 $\therefore$ 主定理失效

e.g. 求递归方程: $T(n) = 3T(n/4) + n \log n$

证: 已知 $a=3, b=4, f(n) = n \log n$

$\therefore n^{\log_b a} = n^{\log_4 3}$

$\therefore f(n) = n \log n = \Omega(n^{\log_4 3 + \epsilon}) \quad (\epsilon = 0.2)$

要使 $a f(n/b) \leq c f(n)$ 成立

$\vee T \quad 3 f(n/4) \leq c f(n)$

$\vee T \quad 3 \cdot \frac{n}{4} \log \frac{n}{4} \leq c \cdot n \log n$

显然只要 $c > \frac{3}{4}$ 即可满足对充分大的 n 成立

这里相当于主定理的第三种情况

$\therefore T(n) = \Theta(n \log n)$

e.g. $\begin{cases} T(n) = 2T(\lfloor \frac{n}{2} \rfloor) + n \\ T(1) = 1 \end{cases}$

证: $\because T(n) = 2T(\frac{n}{2}) + n$ 证为 $\Theta(n \log n)$

$\vee T \quad \exists c > 0, n_0 \text{ s.t. } \forall n \geq n_0, \text{ 有 } T(n) \leq c n \log n$

可证该递推方程中也是 $c n \log n \quad (n \geq 2)$ 归纳证明: 当 $n=2$ 时, $T(2) = 2T(1) + 2 = 4$, 要使 $4 \leq 2c \log 2$, c 可取 2.

$$T(n) = 2T(\lfloor \frac{n}{2} \rfloor) + n \leq 2T(\frac{n}{2}) + n \leq 2c \frac{n}{2} \log \frac{n}{2} + n$$

$$= c n \log n - c n + n \leq c n \log n \quad (c=2)$$

$\frac{n}{2}$ 成立 $\Rightarrow n$ 成立
 $\downarrow$
 一切成立

§4. 分治策略 <上>

核心思想: *Divide* → *Conquer* → *Merge*

显然, 子问题规模一样, 为递归, 为递归更简单.

↳ 时间复杂度:
$$\begin{cases} W(n) = W(|P_1|) + W(|P_2|) + \dots + W(|P_k|) + f(n) \\ W(c) = C \end{cases}$$

非递归代价.

↳ 直接求与规模为 c 的子问题相同, 常通过均匀划分以降低.

案例一. 数值计算优化.

e.g. A. 幂运算 a^n .

分治:
$$\begin{cases} a^n = a^{n/2} \times a^{n/2} \text{ (偶)} \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a \text{ (奇)} \end{cases}$$

$$\begin{cases} W(n) = W(n/2) + O(1) \Rightarrow O(\log n) \\ W(1) = 0 \end{cases}$$

e.g. B. Fibonacci.

定理:
$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

实数累运算推广至矩阵.

$$W(n) = O(\log n)$$

归纳证明.

$n=1$. 为显然的.

假设对 n 命题成立, $\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$

那么 $\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} F_{n+1} + F_n & F_n + F_{n-1} \\ F_n + F_{n-1} & F_{n-1} + F_{n-2} \end{bmatrix}$

$= \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$

对 $n+1$ 成立.

$= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n+1}$

案例二. 最大子数组.

法A: 暴力 (Brute Force). $O(n^2)$.

法B: 分治. → 从中间将数组二分, 最大子数组落在.

递归找两侧 max.

针对 cross 情况, 从中间向两侧扫描, 找 max 前缀 & 后缀, 求和. $O(n)$.

return 三者 max.

$$T(n) = 2T(n/2) + O(n)$$

根据主定理. 类型二, $T(n) = O(n \log n)$.

Left part

Cross midpoint

Right part

△ 两分分治 Alg. 的递归过程:

求值方法:

① $T(n) = \sum_{i=1}^k a_i T(n-i) + f(n)$.

迭代、递归树、尝试.

② $T(n) = aT(n/b) + d(n)$.

迭代、递归树、主定理

§5. 分治策略 <下>

1. 改进分治算法的路径:

$$T(n) = aT(n/b) + f(n) \Rightarrow \begin{matrix} a \downarrow & n/b \downarrow & f(n) \downarrow \\ \text{子问题数} & & \text{非递归部分} \end{matrix}$$

A: 减少子问题个数 a : e.g. Karatsuba 乘法, Strassen 矩阵乘法.

B: 降低合并代价 $f(n)$: e.g. 平面最近点对问题

2. 案例

Case 1: 大整数乘法: calculate 2个 n 位大整数 X 与 Y 的乘积.

a. 普通竖式乘法: $O(n^2)$.

b. 简单分治: $XY = (10^{n/2}A + B)(10^{n/2}C + D)$.

$$= 10^n AC + 10^{n/2}(AD + BC) + BD.$$

$$\therefore T(n) = 4T(n/2) + O(n)$$

$$\Rightarrow T(n) = O(n^2).$$

c. Karatsuba 算法:

$$AD + BC = (A+B)(C+D) - AC - BD$$

$$T(n) = 3T(n/2) + O(n)$$

$$\Rightarrow T(n) = O(n^{\log_2 3})$$

Case 2: 矩阵乘法 (方阵).

a. 直接计算: $O(n^3)$.

b. 简单分治: 将矩阵拆分为 4 个分块矩阵.

$$A \cdot B = C \Rightarrow \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$$\therefore T(n) = 8T(n/2) + O(n^2)$$

$$\Rightarrow T(n) = O(n^3).$$

c. Strassen 算法: 通过 7 个中间矩阵, 使得: $T(n) = 7T(n/2) + O(n^2)$.

$$\Rightarrow T(n) = O(n^{\log_2 7}).$$

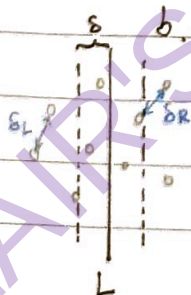
Case 3: 平面最近点对.

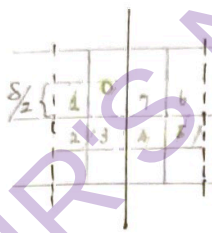
a. 暴力: $O(n^2)$.

b. 分治: ① 沿垂直直线 L 将点集分为 S_L 和 S_R .

② 治: 递归求出两部分内最近距离, $\delta = \min(\delta_L, \delta_R)$.

③ 合: 检查分居两侧的点 (P, Q). $\rightarrow$ 只需检查 L 两侧 δ 区域 (条状区域).





几何性质: 针对条形区域内每个点, 只需 check 其下方点. (理论 max 7个)
(如切面下方也行).

explain: 1. 限制范围 $\left\{ \begin{array}{l} \text{横向: } 2\delta \\ \text{纵向: } \delta \end{array} \right.$

2. The Grid: (如左图), 切成 8 份.

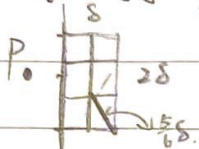
3. each 格子最多 1 个点.

4. 理论上只需考虑另一侧的 4 个, but 为便于 coding, 要 check 7 个.

Moreover, we possess one array Y ordered by y .

So, it's convenient to check the following 7 points without caring about side.

译本的另一思路:



左半在右侧的搜索范围为 $8 \times 2\delta$!

划分为 6 块.

version 1: 每次合并时对区域内点按 y 排序.

因为斜度 $= \sqrt{\delta^2 + (\frac{2\delta}{\delta})^2} = \frac{5}{4}\delta < \delta$, $T(n) = 2T(n/2) + O(n \log n)$ 根据递归树 (经理 X)

保证任一格子中仅 1 个点. $\Rightarrow T(n) = O(n \log^2 n)$

故只需最多 check 6 个点.

version 2 (改进): 将排序工作移出递归, 即增加预处理.

before 递归, 生成 $\left\{ \begin{array}{l} \text{按 } x \text{ 排序的 } P_x \\ \dots P_y \end{array} \right.$

$$T(n) = T_1(n) + T_2(n)$$

$$T_1(n) = O(n \log n)$$

$$\begin{cases} T_2(n) = 2T_2(\frac{n}{2}) + O(n) \\ T_2(n) = O(1), \text{ 当 } n \leq 3. \end{cases} \Rightarrow T_2(n) = O(n \log n).$$

$$\Rightarrow T(n) = O(n \log n)$$

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{O(n)}{n(n+1)}$$

$$\frac{1}{2} S(n) = \frac{T(n)}{n+1}:$$

$$S(n) = S(n-1) + O(\frac{1}{n})$$

$$\Rightarrow \text{调和级数: } S(n) = \sum_{i=1}^n \frac{1}{i} = \ln n$$

$$\therefore T(n) = (n+1) S(n) = O(n \log n).$$

3. 译本上典型应用.

$$\textcircled{1} \text{ 快速排序: } \begin{cases} T(n) = 2T(\frac{n}{2}) + O(n) \\ T(1) = 0 \end{cases} \Rightarrow T(n) = O(n \log n)$$

chⁱ: 子问题规模不一样, but 递推比例. 如 1.9.

$$\begin{cases} T(n) = T(\frac{n}{10}) + T(\frac{9n}{10}) + O(n) \\ T(1) = 0 \end{cases} \xrightarrow{\text{递归树}} \frac{9}{10}^k n = 1 \Rightarrow k = \log_{10/9} n \Rightarrow T(n) = O(n \log n)$$

chⁱⁱ: 极端不平衡.

$$\begin{cases} T(n) = T(n-1) + O(n) \\ T(1) = 0 \end{cases} \xrightarrow{\text{递归树}} T(n) = O(n^2)$$

chⁱⁱⁱ: 平均情况.

$$\begin{cases} T(n) = \frac{1}{n} ((T(0) + T(n-1)) + (T(1) + T(n-2)) + \dots + (T(n-1) + T(0))) + O(n) \\ T(1) = 0. \end{cases}$$

$$nT(n) = 2 \sum_{i=0}^{n-1} T(i) + O(n^2)$$

$$\text{两式相减} \Rightarrow nT(n) = (n+1)T(n-1) + O(n)$$

No.

DATE / /

1. Basis

求最优子问题 $\rightarrow$ 最优解
S6. 动态规划 <上>
 $\hookrightarrow$ 以史为鉴

最优子问题：
和子问题多步判断，从子问题往前，提前求解。
子问题 OPT $\rightarrow$ 原问题 OPT

动态规划 = 递归 + 记忆

DP 是递归的优化

分解子问题，消除重复计算

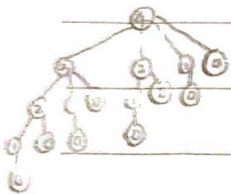
最优子结构 (优化原则)：一个最优决策序列的任何子序列一定是相应子问题的 OPT。
子问题 (空间换时间) $\rightarrow$ 使用 DP 前，先 check 是否满足优化原则。
有些问题，子问题重叠 $\rightarrow$ 整体最优。

2. 案例

Case 1. 钢条切割：给定长度为 n 的钢条，和价格表 P ，求最大收益 r_n 。

a. 暴力搜索：割法有 2^{n-1} 个 (可行解)

b. DP. 第 i 段的价格，长度为 i (最优性)



$$r_n = \max_{1 \leq i \leq n} (P_i + r_{n-i}) \quad // \text{ 设最优解有切割，则可穷举第一刀的可能，} \\ \rightarrow \text{ 则拿部分的最大收益。}$$

$\Rightarrow$ 伪代码: CUT-ROD(P, n)

```
if  $n == 0$ 
    return 0
 $q = -\infty$  // 收益
for  $i = 1 \rightarrow n$ 
     $q = \max(q, P[i] + \text{CUT-ROD}(P, n-i))$ 
return  $q$ 
```

大量重复

$$\Rightarrow T(n) = 1 + \sum_{j=0}^{n-1} T(j) = 2^n$$

$\hookrightarrow$ 改进 A & B

改进: A. 自顶向下备忘 (Top-down with memorization)

递归 + 记忆 memo，每次计算前先查 memo 是否有值。

$\Rightarrow$ pseudocode: ROD-CUT(P, n, r)

```
if  $r[n] > 0$  return  $r[n]$  // 算过的直接回
if  $n == 0$   $q = 0$ 
else
     $q = -\infty$ 
    for  $i = 1 \rightarrow n$ 
         $q = \max(q, P[i] + \text{ROD-CUT}(P, n-i, r))$ 
     $r[n] = q$ 
return  $q$ 
```

pl. 递归 (Top-down)

$\downarrow$ 空间换时间
DP

$\Rightarrow$ 复杂度为平方级

B. 自底向上 (Bottom-up) (推荐)

计算 $r_1 \rightarrow r_n$ ，当计算 r_j 时， $r_1 \rightarrow r_{j-1}$ 已算好，可直接查表。

$\Rightarrow$ pseudocode: ROD-CUT(P, n)

```
 $r[0] = 0$ 
for  $j = 1 \rightarrow n$ 
     $q = -\infty$ 
    for  $i = 1 \rightarrow j$ 
         $q = \max(q, P[i] + r[j-i])$ 
     $r[j] = q$ 
return  $r[n]$ 
```

if $q < P[i] + r[j-i]$ 最优值 $\rightarrow$ 最优解。
 $q = P[i] + r[j-i]$ + 记录切割方案。
 $SE[j] = i$ // 记录切割位置

$\Rightarrow$ 复杂度：平方级

$\hookrightarrow$ 一定更优

从 $1 \rightarrow n$. 从小到大.

No.

DATE

Case 2. 矩阵连乘: $A_1 \times A_2 \times A_3 \times \dots \times A_n$. (A_i 为二维矩阵, 不一定为方阵.)

满足结合律, 不同运算次序代价差异大.

$$A(BC) = (AB)C$$

P: 存储矩阵维度的数组

$$m[i, j] = \min_{i \leq k < j} (m[i, k] + m[k+1, j] + p_i \cdot p_k \cdot p_j)$$

$P[i-1]$: A_i 行数

$P[i]$: A_i 列数
即 A_i 的列数

计算 $A_i \dots A_j$ 的最小代价.

$\Rightarrow$ pseudocode: MATRIX-CHAIN-ORDER(p)

$n = p.length - 1$ // 矩阵数 n .

定义 m 及 s // m : $A_i \dots A_j$ 的最小代价; s : $A_i \dots A_j$ 的最优分割点.

for $i = 1 \rightarrow n$ $m[i, i] = 0$

for $l = 2 \rightarrow n$ // 连乘矩阵的个数. 先算出 all $l=2$ 的矩阵链, 再算 $3, \dots, n$.

for $i = 1 \rightarrow n - l + 1$ // 连乘起点

$j = i + l - 1$ // 连乘终点

$m[i, j] = \infty$

for $k = i \rightarrow j - 1$ // 扫描范围内 all 子问题. 尝试分割点 k .

$$q = m[i, k] + m[k+1, j] + p_i \cdot p_k \cdot p_j$$

if $q < m[i, j]$

$$m[i, j] = q$$

$$s[i, j] = k$$

return m & s .

$$\Rightarrow O(n^3)$$

Case 3. 投资问题. m 元. n 个项目. $f_k(x)$ 为 x 元投入项目 k 的收益. 求最大化总收益.

1. 界定子问题范围: 按项目. 划为 n 个阶段, 按项目 $1, 2, \dots, n$ 依次分配.

同时, 按资金. 划为 m 个方案.

2. 优化函数: $F_k(x)$ 为 x 元投给前 k 个项目获得的最大收益.

(第 k 步时, 已知 p 元 ($p \leq x$) 投给前 $(k-1)$ 个项目的最大收益)

3. 递推方程及边界条件:

$$F_k(x) = \max_{0 \leq r \leq x} \{ f_k(r) + F_{k-1}(x-r) \}, \quad k = 2, 3, \dots, n$$

$$F_1(x) = f_1(x), \quad F_k(0) = 0, \quad k = 1, 2, \dots, n.$$

$$\Rightarrow \sum_{k=2}^n \sum_{x=1}^m ((n+1) + x) \Rightarrow O(n \cdot m^2).$$

穷举 比较.

项目 \ 资金	1	2	3	4	5	...	n
1			0				
2			0				
3			0	0			
4							
5							
...							

$F_4(3)$

$F_2(4)$

$F_4(4)$

§ 7. 动态规划 <下>

1. 解题流程:

① 刻画最优子结构: 证明原问题最优解, 包含子问题最优解.

② 递归定义最优值: 写状态转移方程. (注意边界条件).

③ 计算最优值: Bottom-up 代码.

④ 构造最优解: 利用辅助数组回溯.

2. 案例:

Case 1. 最长公共子序列 (LCS)

给定: $X = \langle x_1, x_2, \dots, x_m \rangle$, $Y = \langle y_1, y_2, \dots, y_n \rangle$.① 暴力: 又有 2^m 个子序列. 每次检查 $O(n) \Rightarrow O(n \cdot 2^m)$.② DP: 关注其中的选择: whether to select the last one of X & Y .

$$C[i, j] = \begin{cases} 0 & i=0 \text{ or } j=0 \\ C[i-1, j-1] + 1 & x_i = y_j \\ \max\{C[i-1, j], C[i, j-1]\} & x_i \neq y_j \end{cases}$$

X 的前 i 个 和 Y 的前 j 个
字符的 LCS 长度.

若 $x_i = y_j$ 则最后字符相同
 则 LCS + 1, 包含这个字符
 else, $x_i \neq y_j$ 则 -.

pseudo code:

LCS(X, Y): $m = X.length; n = Y.length;$ // 长度.for $i \leftarrow 0$ to m do $C[i, 0] = 0$ > when $i=0$ or $j=0$ $C=0$ for $i \leftarrow 0$ to n do $C[0, i] = 0$ for $i \leftarrow 1$ to m do// $i: 1 \rightarrow m$ for $j \leftarrow 1$ to n do // $j: 1 \rightarrow n$ if $X[i] == Y[j]$ then $C[i, j] = C[i-1, j-1] + 1$ $B[i, j] = \nwarrow$ else if $C[i-1, j] \geq C[i, j-1]$ then $C[i, j] = C[i-1, j]$ $B[i, j] = \uparrow$

else

then $C[i, j] = C[i, j-1]$ $B[i, j] = \leftarrow$

e.g.

 $X = \langle A, B, C \rangle, Y = \langle B, D, C \rangle$

$X \backslash Y$	0	1	2	3
0	0	0	0	0
1	0	0	1	1
2	0	1	1	1
3	0	1	1	2

 $Z = \langle B, C \rangle$

到可能矩阵. 2维.

No

DATE / /

Structure Sequence (B, i, j): // 标记并输出.

if $i=0$ or $j=0$ then return;

if $B[i, j] = '\swarrow'$

then output (Z[i])

Structure Sequence (B, i-1, j-1)

else if $B[i, j] = '\uparrow'$

then Structure Sequence (B, i-1, j)

else // ' $\leftarrow$ '

then Structure Sequence (B, i, j-1)

$\Rightarrow O(mn)$

Case 2. 0-1 背包问题. $\rightarrow$ 类似投资问题.

给定: n 个物品. 重量 w_i . 价值 v_i . 背包容量 b . 求最大价值.

选择: whether to put in the k -th item.

$$F_{k,y} = \max \{ F_{k-1,y}, F_{k-1,y-w_k} + v_k \}$$

$\nwarrow$ 前 $k-1$ items, 重量 $\leq y$ 的最大价值

$\rightarrow$ 选 k -th

$\rightarrow$ 选.

时间复杂度 = 表格物数 $\times$ 单个计算时间 $= (n \times b) \times O(1) \Rightarrow O(nb)$

空间 $\dots: O(nb)$

$F_{k,y}, k \in 0 \rightarrow n, y \in 0 \rightarrow b$.

$\hookrightarrow$ 伪多项式时间算法.

0-1 背包: 对每个 item, only '选' or '不选' 2 choices $\rightarrow O(1)$

投资: 对每个 item, 有 $0 \sim m$ 的 m choices $\rightarrow O(m)$

$\Rightarrow$ 状态转移方程不同 $\Rightarrow O(nb)$

$\rightarrow$ 连续. $O(nm^2)$

Case 3. 最大子数组和.

给定: 数组 $A[1..n]$. 找子数组 $A[i..j]$. 使和最大.

① 暴力: $O(n^2)$

② 分治: $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \log n)$

③ DP:

$$C[i] = \max \{ C[i-1] + A[i], A[i] \}$$

$$\Rightarrow \text{OPT}(A) = \max \{ C[i] \}$$

$$T(n) = O(n)$$

$\hookleftarrow A[i] \rightarrow A[1..n]$

要满足优化原则,

定义 $C[i]$ 为包含 $A[i]$

才能与后续 $A[i+1]$

连接上.

No.

贪心选择性质: 所求问题的整体 OPT 可由一般局部 OPT (贪心选择) 来得到.

DATE / /

§8. 贪心法 <上>

1. 案例: 活动选择问题: 找最大互不冲突子集.

o 贪心策略: 结束时间早优先.

o 关键要素:

(1) 贪心选择性质: 做出局部最优选择, 一定能导致全局最优.

证: (剪切-粘贴法).

假设一个最优解 A_k , 将第一个活动非贪心选择的 a_m , 则用 a_m 替换第一个 a_j .

依然可行: a_m 比 a_j 结束早, so 不会与 others 冲突.

依然最优: 集合大小不变.

$\therefore$ 贪心选择安全.

(2) 最优子结构: 贪心选择后, 子问题最优 + 已选 = 原问题最优.

2. 一般步骤.

o 思路: 通过贪心选择改进最优子结构, s.t. 选后仅剩一个子问题.

① 将优化问题转化 $\rightarrow$ 选后剩单个子问题.

② 安全: 满足性质

③ 满足最优子结构

对最优子结构的归纳.

贪心选择性质:

贪心策略是一个递归

的过程, 需要证明

第1步正确, 后面也OK.

△ 正确性证明:

设 S 为活动集合, 已按结束时间排好序 ($t_1 \leq t_2 \leq \dots \leq t_n$). 要证: 子集-OPT. A include 贪心的

① 归纳基础: 设 A 为原问题一个最优解, 其中结束最早的是活动 k .

证 $k=1$, 命题得证.

证 $k \neq 1$, 已知 $t_1 \leq t_k$, 可构造新解 $A' = (A - \{k\}) \cup \{1\}$, 也不会产生冲突.

故 $|A'| = |A|$, A' 也是 OPT, 命题得证.

$\Rightarrow$ 贪心选择第1步是安全的.

④ 结论: 由数学归纳法,

② 归纳假设: 假设贪心算法前 k 步的选择 ($i_1, i_2, \dots, i_k$) $\in$ 某 OPT.

贪心算法每一步选择

③ 归纳步骤: 定义子问题 S' 为与前 k 个活动相容的剩余活动集合.

$\in$ 某 OPT 中, 因此最

终解为 OPT.

根据最优子结构性质, 原问题 OPT = 前 k 个活动 + S' 的 OPT.

可证明性: 第 $k+1$ 步为 S' 中选择结束时间最早的. 重复归纳步骤中的逻辑, 所选活动 $\in S'$ 的 OPT 中. 因此, 加上前 k 个, 这些构成全局 OPT 的一部分.

§9. 贪心法 <下>

1. 典型应用:

Case 1: Huffman 编码. 解决无损压缩. 得到最优前缀码.

贪心选择: 找权值最小的 2 棵树合并.

正确性证明:

① 引理 1 (性质): 频率 $\min$ 的 2 字符 x, y , 在最优树中为兄弟节点, 且在最底层.证: 设 x, y 非兄弟, 则可进行 swap, 后带路径长度 WPL 不会 ↑.② 最优子结构: $f(x) + f(y) = f(z)$. 原优 = 子优 + $f(z)$.? 合并后的子问题最优解 T' , 那么还原回去的解 T 为原优吗? $n=2$ 成立.

证: (反证法).

 $n=k \rightarrow n=k+1$ C : 原符号集. C' : 合并后符号集. $z \rightarrow xy$ (别称)

↓ 合并.

反设: T 非原优. 故存在 T' 为原优. $WPL(T') < WPL(T)$.由引理 1, 可合并 T' 得到 T'' (x, y 为 $\min$)反证法核心: T 非 OPT. 故 $WPL(T'') + (f(x) + f(y)) < WPL(T') + (f(x) + f(y))$ 则 T' 为 OPT. $WPL(T'') < WPL(T')$ 合并 T' 中的 x, y .得到 T'' . 优于 T' .与 T' 为子问题 OPT 矛盾. $\therefore T$ 为原优.

Case 2. 最小生成树.

原图 $G=(V, E)$: n 顶点, m 条边.A. Prim 算法: 初始 $\{v_0\}$. 选邻接权最小的边加入 (边的另一端点为已连接点集 U).另一端点为 $V-U$ 中).

正确性证明: 反证 + 剪枝 + 粘贴.

设最优树不含要加入的最短边 e , 则加 e 会多生回路.而回路中 $\exists e' > e$, 则可将 e' 用 e 替换, 结果更优.

B. Kruskal 算法: 不断 add 最短且不成回路边.

→ 证明

证: $n=2$ 成立. $n=k$. 可得到 MST.

↓

→ 中最大权边. → $n=k$. $n=k+1$. 连接 $e \Rightarrow G'$. T 为 G' 的 MST. $\Rightarrow T = T' \cup \{e\}$.(反证法) T 非 MST. $\exists T''$. ① 确保 e 在 T'' 中. ② 短接 e . $W(T'' - \{e\}) < W(T')$ $\therefore T \neq \text{MST}$

矛盾

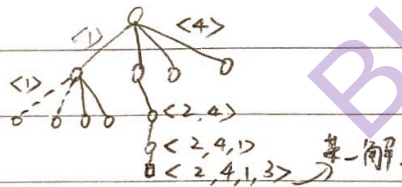
1. 回溯法: *backtracking*

- *AO*: 一种遵照某种规则 (*avoid 遗漏*)、跳跃式 (*剪枝*) 地搜索解空间的技术.
- *Core*: DFS + 剪枝.
- 适用性: 搜索问题 (找可行性) & 优化问题 (找最优解).

2. 关键建模要素:

① 解: $\langle x_1, x_2, \dots, x_n \rangle$ ② 解空间: the set of all candidate solutions, 通常树形表示. 是否满足激活条件③ 约束条件: 剪枝依据. 确定搜索路径.④ 目标函数: ^{constraint} 评估解的优劣.

3. 案例.

Case 1: *N*皇后.① 解: $\langle x_1, x_2, \dots, x_n \rangle$. x_i 表 i -皇后放置皇后的列号.② 解空间: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid 1 \leq x_i \leq n, i=1, \dots, n \}$ ③ 约束: 列: $x_i \neq x_j, i \neq j$.斜线: $|x_i - x_j| \neq |i - j|, i \neq j$ 23妙啊!!e.g. 4皇后 $\rightarrow$ 构造4叉树. $\Rightarrow$ *N*皇后: *N*叉树.最坏情况: $O(n^{n+1})$.

Case 2: 0-1 背包.

① 解: $\langle x_1, x_2, \dots, x_n \rangle$. $x_i = 1$ 表 i -th 物品被选中.② 解空间: $\{0, 1\}^n$ ③ 约束: $\sum w_i x_i \leq b \rightarrow$ 超重则剪枝.④ 目标函数: $\max \{ \sum v_i x_i \}$ 最坏: $O(2^n)$.

Case 3: 旅行商 (TSP): 经过 all cities 并回到起点的 Hamiltonian circuit.

① 解: $\langle l_1, l_2, \dots, l_n \rangle$ 为路线. 设 $l_1 = 1$.

② 解 space: 以 1 开头的 $1 \sim n$ 的排列.

③ no constraints

④ 目标函数: $\min \{ \sum l(i_j, i_{j+1}) + l(i_n, i_1) \}$

最坏: $O((n-1)!)$.

4. 回溯法适用条件.

△ 多米诺性质: 若部分解 $P(x_1, \dots, x_k)$ 不满足约束.

Dominance property.

then all 扩展解 $P(x_1, \dots, x_{k+1})$ 均不满足.

↓
若不满足多米诺性质, 须先进行变量变换使其满足. (正确解时可能被剪枝)

e.g. $5x_1 + 4x_2 - x_3 \leq 10, 1 \leq x_3 \leq 3$.

令 $x'_3 = 3 - x_3, 0 \leq x'_3 \leq 2$

$\Rightarrow 5x_1 + 4x_2 + x'_3 \leq 13$

5. 蒙特卡洛 (Monte Carlo) $\rightarrow$ 估算树的结点数, 即实际搜索空间. $\rightarrow$ 效率估计.

△ 方法: 从根结点, 采用随机采样对 x_j 赋值, 直至得到一完整路径.

利用结果 $\rightarrow$ 无偏估计量 N .

重复采样, 取 AVG, 根据大数定理, 得到估计值.

△ 计算: 某结点有 k 个分支, 随机选 j 个分支, 得到估计值 N_j .

$\Rightarrow$ 树结点估计值: $k \times N_j + 1$.

△ 递归实现模板代码:

△ (伪代):

Backtrack(k):

if $k > n$ then output 解向量 x .

for val in S_k : // 遍历 k 层 all candidates

$x[k] = \text{val}$

if constraint(k): // 剪枝

Backtrack($k+1$) // 进入下一层.

$k = 1$; 计算 S_1 .

while $k > 0$:

if S_k 非空:

then $x[k] = S_k.\text{pop}()$

if $k = n$: output(x)

else: $k++$; 计算 S_k

else: $k--$

(回溯后, 确定是 S_{k+1}).

§ 11. 分支限界.

1. 分支限界:

△剪枝依据

- 约束条件
- 地位与边界
- 最大化
- 最小化

- Q: 用于求解 组合优化问题 的算法。
- 机制: Based on 回溯, during DFS, 引入 额外约束(界) 来剪枝。
- Core: 如果一个部分性决策如何扩张, 都不能 surpass 已有 OPT. 则剪枝。
- Concepts: _____

● 累：已找到最优解时的目标函数值

key 代理函数: 对子树最优结果的估计

判定法则: (max 问题) 当前 node 的估算上界 $UB <$ 已知最大值 LB ,
(min 相反) 则剪枝.

2. 案例:

Case 1: 0-1 背包.

① preprocess: sort by (v_i/w_i) DESC.
單位重量價值.

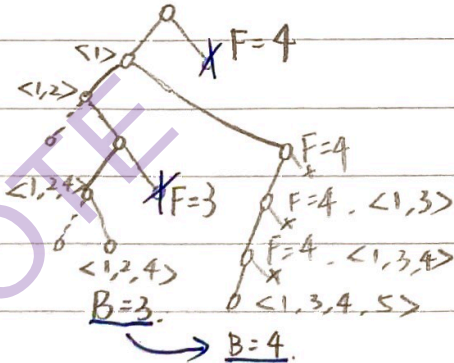
② 代价承股 (VB):

$$UB = \text{已装入价值} + \text{剩余空间} \times \text{next item 单位价值}$$

Case 2: 最大团问题: 求顶点数最多的完全子图.

$UB = \text{当前团内顶点权} + \text{剩余顶点权}$

e.g.  解: $\langle v_1, v_2, v_3, v_4, v_5 \rangle$ 中 v_i 表示 i 号顶点是否入团。
递归: (左选 1: 入, 右选 0: 不入)



F : cost function.

B: bound.

Case 3. TSP. Travelling salesman problem.

o type: minimize.

o ① $LB = \text{当前路径长度} + \text{最短出边} + \text{剩余顶点最短出边和}$

压缩搜索空间

$$B = \{1, i_2, \dots, i_k\} \quad \min_{l \neq i_k} \{i_k, l\} \quad \sum_{j \in B} \min_{l \neq j} \{j, l\}$$

② $LB = L_{curr} + i_k \text{最短出边} + 1 \text{最短出边} + MST\{V-B\}$

$$\min_{l \in B} \{i_k, l\} \quad \min_{l \in B} \{1, l\} \quad V-B \text{ 的最小生成树}$$

Case 4. 连续邮资问题: n 个面值, 最多 m 张, 求最大连续区间.

1. 初始: 面值 $a_1 = 1$.

2. 计算当前能力: m 张限制下, 最大值 r_i .

3. 分支: 下一个取 $a_i + 1 \sim r_i + 1$

4. 递归

5. 更新: 统计节点, 比较 m , 更新 OPT.

§12. 问题复杂性.

1. Basis.

问题复杂度: 一个问题的最高效解决算法的复杂度.

复杂度 $\Theta(f(n))$ $\leftarrow$ 可达性: $\exists$ 复杂度为 $O(f(n))$ 的算法. $\rightarrow$ 最好情况下的复杂度.
 下界: $\forall$ 算法复杂度为 $\Omega(f(n))$.

算法上界 O 与问题下界 Ω 重合 Θ $\Rightarrow$ 渐进最优算法.

2. 确定下界的方法.

① 平凡下界: 根据输入数据量或输出数据量确定.

e.g. 1. 写出 all n 阶置换 $n!$ 个解 $\rightarrow \Omega(n!)$

通常过低
而先定时间 e.g. 2. 多项式求值 n 个输入参数 $\rightarrow \Omega(n)$

e.g. 3. 矩阵乘法 输出 $n \times n$ 个值 $\rightarrow \Omega(n^2)$

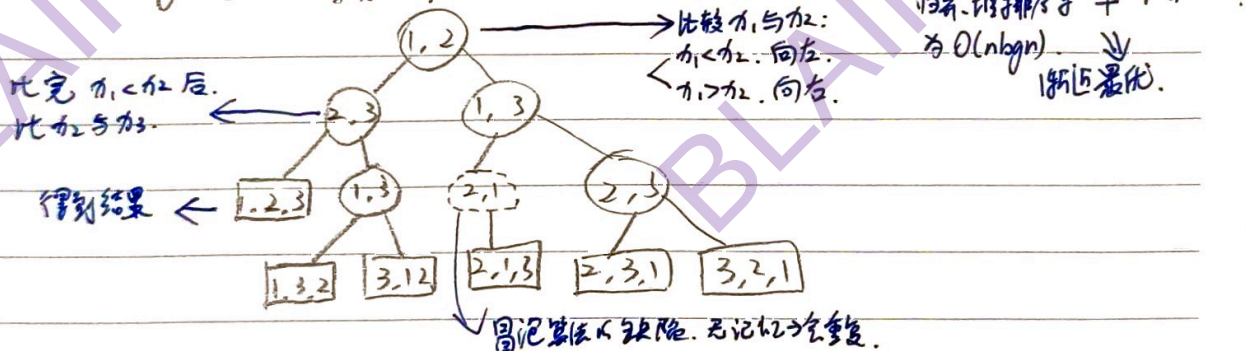
(count 基本运算数).

② 决策树: 以比较为基础运算.

模型: 二叉树, 内部节点为一次比较, 叶节点为结果.

定理: 任何基于比较的排序算法在最坏情况下需比较 $\lceil \log_2 n! \rceil \approx n \log n$ 次.

e.g. 冒泡: 输入 $\{n_1, n_2, n_3\}$.



定理证明: 最坏比较次数为树高 h .

叶结点数 (结果数): $n!$ (A_n^n).

树高为 h 的叶结点数至多: 2^h (二叉完全树).

$$\Rightarrow 2^h \geq n!$$

$$h \geq \log_2 n! \approx n \log n.$$

③ 对于结论: 为算法构造最好输入.

e.g. 1. Find Max.

o 下界: $(n-1)$ 次比较.

o 理由: n 个数中取 max, 需淘汰 $(n-1)$ 个, 每次比较最多淘汰 1 个.

e.g. 2. Find Max & Min.

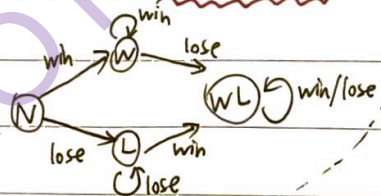
o 下界: $\lceil \frac{3n}{2} \rceil - 2$ 次比较.

o 最优策略: ① 两两比较, 分 Win 组 & Lose 组, $(\frac{n}{2})$

② Win 中找 Max, Lose 中找 Min. $(\frac{n}{2}-1) \times 2$

$\Rightarrow \text{total} \approx \frac{3}{2}n$.

o 思路: (状态机分析)



o 信息单位:

N: 0

W/L: 1

WL: 2

$\Rightarrow \text{start: } 0$

$\text{end: } 2n-2$ (must meet).

ii

得分方式:

$N \text{ vs } N + 2$
 $N \text{ vs } W/L + 1$
 $W \text{ vs } W/L \text{ vs } L + 1$
 $WL \text{ vs } WL + 0$

iii Δ 计算极限: $(2n-2)/2 = n-1$ 次.

$\rightarrow$ 理想情况下, 每次 +2.

iv Δ $\frac{n}{2} \times 2 = n$ 分.

$\rightarrow$ 实际最多 +2 次

$[2n-2] - n = n-2$ 次

$\rightarrow$ 还需 +1.

$\Rightarrow \frac{n}{2} + (n-2) = \frac{3}{2}n - 2$ 次.

④ 规约: 利用已知问题的难度 (证明新问题的难度).

问题: $Q \xrightarrow{\text{reduction}} P$ ($Q \leq P$).

实例: $I \longleftrightarrow A(I)$.

下界: $\Omega(f(n)) \rightarrow$ 至少 $\Omega(f(n))$ 已知.

o 推导流程: 1. Q hard.

2. $Q \xrightarrow{\text{reduction}} P \xrightarrow{P \text{ 的解}} P \text{ 的解} \rightarrow Q \text{ 的解}.$

3. if P easy $\rightarrow Q$ easy. 矛盾

$\therefore P$ also hard.

(为了证明)

P 的问题难度 $> Q$ 的.
 Q 的下界 $\rightarrow P$ 的下界.
 Q 的已知 $\rightarrow P$ 的已知.

No.

DATE / /

e.g. 1. 因子分解 $\text{factor}(n)$ P
素数测试 $\text{testP}(n)$ Q

假设 Q 复杂度 $W(n)$.

Q 算法: 1. $p \leftarrow \text{factor}(n)$

2. if p then return false

3. else return true

$\Rightarrow \Omega(W(n)) = T_Q \leq T_P$.

B // 写完作业后, 必然知道了有无作业.
A $\Rightarrow T_A \leq T_B$. (In fact, 先完成 A, 再完成 B.)

e.g. 2. 已知: 元素唯一性问题: 集合 S 中有 n 个元素. $W(n) = \Omega(n \log n)$.

问题: 最坏点 P

4. 变换: Q 的实例: $n_1, n_2, \dots, n_n \xrightarrow{f} (n_1, 0), (n_2, 0), \dots, (n_n, 0)$.

Q: 1. 用 P 算法求最短距离 d .

2. if $d=0$ then return false

3. else return true

$\Rightarrow$ 若 P 能更快解决, 那 Q 也能. 与 Ω 矛盾. $\therefore$ 不能 $\Rightarrow T_P = \Omega(n \log n)$

§ 13. NP完全性.

1. Basis.

判定问题

P类: 多项式时间可解.

polynomial

NP类: 多项式时间可验证.

nondeterministic polynomial

$$P \subseteq NP$$

$$NPC \subseteq NP\text{-hard}$$

多项式时间归约: $A \leq_p B$. 表示可多项式归约至 B . $\exists$ 多项式时间算法 f 使 A 任一实例 $I \rightarrow B$ 的实例 $I' = f(I)$.

具有传递性

$$I \in \text{Yes}(A) \iff I' \in \text{Yes}(B)$$

e.g. $HCS \leq_p TSP$

证: 对 HCS 任一实例 $I: G = \langle V, E \rangle$, TSP 对应的实例 $f(I)$ 为:

城市集 V , $d(u, v) = \begin{cases} 1, & \text{if } (u, v) \in E \\ 2, & \text{else} \end{cases}$, 界限 $D = |V|$.

$\Rightarrow$ 可在多项式时间计算

$\searrow$ u, v 不同, TSP 问题解决的解是哈密顿回路. $\Rightarrow$ 哈密顿回路 $\Rightarrow$ 城市子排列. $\Rightarrow$ 哈密顿回路 $\Rightarrow$ 城市子排列.

$$I \in HC \iff f(I) \in TSP$$

Δ 若 $A \leq_p B$, $B \in P \Rightarrow A \in P$.

证: 设 f 是 $A \rightarrow B$ 的多项式时间归约.

$\because B \in P \therefore \exists B$ 的多项式时间算法 g .

$\Rightarrow A$ 的结果为 $g(f(I))$. 为多项式时间算法.

$\therefore$ 得证.

Δ NP完全性 (NPC) nondeterministic polynomial complete.

Δ NP-hard: all NP 可 $\leq_p$ NP-hard. $\Rightarrow$ NP难. 不一定是 NP. 只是 all can 归约到它.

$$NPC = NP \& NP\text{-hard}$$

GNP

ENP-hard

不定GNP

$\leftarrow$ NPC: 难. 能够归约.

$\leftarrow$ NP-hard: 难. 不一定能够归约.

$\star \Delta$ 证明是 NPC 的方法: (1). 证明 $\pi \in GNP$

(2). 找一个已知解 NPC 问题 π' . 证明 $\pi' \leq_p \pi$. $\Rightarrow \pi$ 是 NP 难的.

2. 案例.

e.g. 1. 可满足性 (SAT) 问题: 给定一个合取范式 F , 判断是否可满足. (是否所有成真赋值).

特例: 3-SAT

Δ 特例: 若 π 的 all 实例均为 π' 的实例. 则 π 为 π' 的特例.

$\Rightarrow$ 性质:

$$\text{if } \pi' \in P \rightarrow \pi \in P$$

$$\text{if } \pi' \in NP \rightarrow \pi \in NP$$

(逆命题均不成立).

No.

DATE / /

→ 每个简单析取式由3个文字组成的合取范式。

e.g. 1.1. 证明 3-SAT 是 NPC.

证: step 1. 3-SAT $\in$ NP.

$\therefore$ SAT $\in$ NP $\therefore$ 3-SAT $\in$ NP.

step 2. 3-SAT $\leq_p$ 3-SAT.

任给 SAT 的 F, 生成 3-SAT 的 F'.

→ 转换: F 中 $C_j = l_1 \vee l_2 \vee l_3 \vee \dots \vee l_k$

$k=1$: $C_j = l_1 \vee l_1 \vee l_1$

$k=2/3$ 同理扩充.

$k=4$: $C_j = (l_1 \vee l_2 \vee \neg \alpha) \wedge (l_3 \vee l_4 \vee \alpha)$

→ $C_j = (l_1 \vee l_2 \vee \neg \alpha) \wedge (\neg \alpha \vee l_3 \vee \neg \alpha) \wedge \dots \wedge (\neg \alpha \vee l_{k-1} \vee \neg \alpha)$

理由: l_i 为真, 则其他括弧中均为真.

e.g. l_i 为真, 则 $\alpha \rightarrow \neg \alpha$ 为真. (前) $\rightarrow$ 保证其他括弧
为真 $\rightarrow$ 为假 (后) 不影响合取结果
为真.

$\therefore$ F 可在多项式时间内生成.

$F \in \text{SAT} \Leftrightarrow F' \in \text{3-SAT}$.

$\therefore$ SAT $\leq_p$ 3-SAT.

综上, 3-SAT 是 NPC.

→ max 完全图.

e.g. 2. 最大团问题 (CLIQUE): 给定 $G=(V, E)$ 和 $k \in \mathbb{N}^+$, 问: G 中是否存在大小为 k 的团?

step 1. 显然, CLIQUE $\in$ NP.

step 2. 3-SAT $\leq_p$ CLIQUE.

令 $\Phi = C_1 \wedge C_2 \wedge \dots \wedge C_r$ 为 3-SAT 输入, 其中 $C_i = (l_i^1 \vee l_i^2 \vee l_i^3)$

构造图: 1 顶点: each C_r 在 G 中构造 3 个顶点 v_r^1, v_r^2, v_r^3 .

(共 $3R$ 个顶点, each 对应 Φ 中一个文字位置).

构造边: $C_R \times O(1)$

$\Rightarrow O(R^3)$.

$\therefore$ 构造是在多项式时间内的.

2. 团大小为 k .

正确性证明: 必要性 ($\Rightarrow$): 团大小为 $k \Rightarrow$ 有团.

可在 Φ 每个子句中找到一个为真的文字, 共 k 个顶点

满足 (A)

均满足 (B).

充分性 ($\Leftarrow$): G 有团打团 $\Rightarrow$ 重引满足

由于同一句顶点互不相邻, 每个顶点属于一个互不相交的团中, 每个团中各取一个.

真值赋值: 将每个文字赋真值, 其余任意赋值.

→ 该文字不会出现逻辑冲突 (无矛盾同时出现)

$\therefore$ 使得 each 子句至少有一个成真文字, 得证.

§ 14. 难问题处理策略.

△ NP-难 是基于最坏情况定义的. 实际应用中. 往往没有那么极端.

△ 处理策略:

精确求解

高效的指数级算法^①

弱化要求

随机算法^②、近似算法^③、伪多项式时间算法^④

启发式搜索^⑤

基于经验/局部改进, 无严格质量保证.

注解: ① When n 较小, 代价可接受.

② 用正确率换速度. 过程中有随机选择. (而非遵循固定的确定性规则).

e.g. Monte Carlo 和 Las Vegas.

③ 不追求最优值.

④ e.g. 0-1 背包: NPC. 复杂度 $O(nb)$. $\Rightarrow$ 当 b 较小时, 算法高效.

⑤ 近似率.

$$\alpha_A = \max_I \frac{A(I)}{OPT(I)}$$

→ 最坏情况.

e.g. 最小点覆盖问题 (Min-VCP). NPC.

通过极大匹配问题求解:

点覆盖: cover all edges 的顶点集.

1. 找出一组极大匹配 A (互不相邻的边). pl : 匹配: 一组边中.

任两边无公共端点 (分散).

2. Add all 边的端点 into set C .

3. C 为一个点覆盖. 且为 $\frac{2}{\alpha}$ -近似的.

4. 证明思路: 为 cover A 中边, OPT 至少 need $|A|$ 个点;

而 our 算法取了 $2|A|$ 个点. $\Rightarrow$ 误差 ≤ 2 倍.

证明: $\because A$ 极大. 假设边 v 未被 C cover.

则其 2 端点都不在 A 中. 那其本可以加入 A .

故矛盾.

证明.

⑤ 局部搜索: 定义邻域映射. 只走向比当前解更优的邻居.

当邻域内无更优时. stop & output local optima.

缺点: 1. 局部最优而不能保证全局最优.

2. 依赖初始值.

§ n. 补充例题.

[DP]

e.g. 1. 图像压缩 $P = \langle p_1, p_2, \dots, p_n \rangle$, 其中 p_i 为第 i 个像素的值 ($0 \sim 255$).采用分段压缩, 如第 i 段, 有 l_i 个像素, each pixel 用 b_i 位.↳ 段头信息额外需 $8+3=11$ 位.1. 子问题的划分: 子问题树 $P_i = \langle p_1, p_2, \dots, p_i \rangle$.

2. 递推关系:

↳ P_i 的最优分段为 $\rightarrow$ 进制位段.

$$\begin{cases} SC[i] = \min_{1 \leq j \leq \min\{i, 256\}} \{ SC[i-j] + j \times b[i-j+1, i] + 11 \} \\ SC[0] = 0 \end{cases}$$

最后一段所需最大位长.

3. 计算最优值:

伪代码: $Compress(P, n)$:输入: 数组 $P[1..n]$ 输出: 最小位长 $SC[n]$, $l[1..n]$.

全局 init.

$Lmax = 256$ // 最大段长.

$header = 11$ // 段头信息

$SC[0] = 0$

for $i \leftarrow 1$ to n do // 从 $SC[1] \rightarrow SC[n]$, bottom-up.记录 $b[i] \leftarrow \lfloor b[i] \rfloor \leftarrow \text{length}(P[i])$ 最后一段 $bmax \leftarrow b[i]$ 最优值 $\leftarrow SC[i] \leftarrow SC[i-1] + bmax$ 最后一段长度 $\leftarrow l[i] \leftarrow 1$

init.

为提供
前面 $SC[i-1]$ for $j \leftarrow 2$ to $\min\{i, Lmax\}$ do // 最后一段含 j 个像素.

if $bmax < b[i-j+1]$ // 更新 $bmax$

then $bmax \leftarrow b[i-j+1]$

if $SC[i] > SC[i-j] + j \times bmax$ // 更新 $SC[i]$

then $SC[i] \leftarrow SC[i-j] + j \times bmax$

 $l[i] \leftarrow j$ $SC[i] \leftarrow SC[i] + header$ 4. 构造最优解: $Traceback(n, l)$: $j \leftarrow 1$ while $n \neq 0$ do $C[j] \leftarrow l[n]$ $n \leftarrow n - l[n]$ $j \leftarrow j + 1$ $\Rightarrow O(n)$.

No.

DATE / /

[贪心]

e.g. 2. 有几个不同重量的物品, 重量 w_i , 背包容量 C , 要装最重, 最大化.

贪心策略: 重量优先.

正确性证明: 对规模归纳.

① 归纳基础: 当 $n=1$, $w_1 \leq C$, 装入, 显然为 OPT.

② 归纳假设: 假设当 $n=k$ 时, 贪心可得 OPT.

③ 归纳步骤: 考虑 $k+1$ 个的情况, $w_1 < w_2 < \dots < w_{k+1}$.

贪心选择 w_1 .

子问题: $N' = \{2, \dots, k+1\}$, 剩余容量 $C' = C - w_1$.

根据归纳假设, 贪心可在 $\langle N', C' \rangle$ 上找到 OPT, I' .

全局 OPT, $I = I' \cup \{1\}$.

反证法: 假设 $\exists$ OPT, O , 且 $|O| > |I|$, 证明会存在, 整体为 OPT.

若 $1 \in O$, 则 $O - \{1\}$ 也为 $\langle N', C' \rangle$ 的一个 OPT.

且 $|O - \{1\}| > |I| - 1 = |I'|$, 这与 I' 为 OPT 矛盾.

若 $1 \notin O$, 因为 1 是最轻的, 可用 1 去 replace 任一箱.

得 $1 \in O'$ 且 $|O'| = |O|$, 与上一种情况一致矛盾.

④ 结论: 由数学归纳法, 该贪心对任意规模 n 均成立.

[贪心]

e.g. 3. 最小化最大延迟: 一堆任务, 耗时 t_i , deadline d_i .

贪心: 最早截止时间优先.

正确性证明: 交换论证.

S 为贪心调度 ($d_1 < d_2 < \dots < d_n$), 设 O 为某 OPT.

① 逆序对: 若 S 与 O 不同, then O 中必有 2 相邻任务 i 与 j ,
 i 在 j 前, but $d_i > d_j$.

② 交换: 交换 O 中 i 与 $j \Rightarrow O'$.

交换后, 于提前完成, 其延迟时间不会 ↑ Lateness.

$$\begin{array}{l} \text{before: } f_j = s + t_i + t_j, \\ \text{after: } f_i = s + t_j + t_i, \end{array} \quad \left. \begin{array}{l} f_j = f'_j, \\ f_i = f'_i, \end{array} \right\} \begin{array}{l} L_j = f_j - d_j, \\ L_i = f_i - d_i, \end{array} \quad \left. \begin{array}{l} \therefore d_j \\ \hat{d}_i \\ L_j \\ L_i \end{array} \right\}$$

结论: $L_{\max}(O') \leq L_{\max}(O)$,

through finite swap, we can $O \rightarrow S$, 且 $L_{\max}$ 不会增加.

$\therefore S$ 为一个 OPT.

[贪心]

e.g. 4. Prim 证明: (反证法)

假设进行到 step k . 顶点集 S . 选择最大权值边 e_k .① 假设 G 的任-MST 含 e_k . T^* 为某-MST.② 构造回路: add e_k to T^* . 必会多出回路.③ 寻找替换边: e_k 两点在 S 与 $V-S$ 中. $\rightarrow$ 该回路中含另一连接两集合的边 e .④ 权值比较: $w(e_k) \leq w(e)$.⑤ 交换: 新树 $T' = (T^* - \{e\}) \cup \{e_k\}$. $\rightarrow$ 连通性 $\checkmark$
 $\rightarrow$ 权值: $W(T') \leq W(T^*)$ ⑥ 结论: cause T^* is MST, $W(T') = W(T^*)$.so T' is also MST. & includes e_k .

综上, 贪心的 each choice is safe. 在 OPT 中.

[回溯]

e.g. 5. 装载问题: 两船载重 C_1 & C_2 . 有 n 个集装箱, 重 w_i . $\sum w_i \leq C_1 + C_2$.w/p: 解题思路: 用回溯定第1条船最大载重 W_1 . $T \min(C_1 - W_1)$.if $\sum w_i - W_1 \leq C_2$. 则 Yes.

else No.

算法设计: (1). 待问题 $\langle n, \langle w_1, w_2, \dots, w_n \rangle \rangle$. 其中 $w_i \in \{1, 0\}$. $1 \leq i \leq n$. 搜索空间为子集(2). 约束条件: $\sum_{i=1}^n w_i x_i \leq C_1$.(3). 满足可行性性质: $\sum_{i=1}^n w_i x_i > C_1 \Rightarrow \sum_{i=1}^{k+1} w_i x_i > C_1$.

(4). 搜索策略: DFS.

伪代码:

Loading (W, C_1):输入: 集装箱重 $W = \langle w_1, w_2, \dots, w_n \rangle$. C_1 为船1载重.输出: 船1最大载重量 $\langle x_1, x_2, \dots, x_n \rangle$. 其中 $x_i \in \{0, 1\}$. $B \leftarrow C_1$; best $\leftarrow C_1$; $i \leftarrow 1$.while $i \leq n$ do

A:

if 装入后超重

then $B \leftarrow B - w_i$; $x[i] = 1$; $i \leftarrow i + 1$.else $x[i] = 0$; $i \leftarrow i + 1$.if $B < \text{best}$ then 记录并; best $\leftarrow B$; $i \leftarrow i - 1$.Backtrack (i)if $i = 1$ then return 最优解

else goto A

Backtrack (i)return while $i > 1$ & $x[i] = 0$
至父节点 $i \leftarrow i - 1$.reach 右分支. if $x[i] = 1$ then $x[i] \leftarrow 0$ $B \leftarrow B + w_i$ $i \leftarrow i + 1$